

Chapter 9 — Components, Page, and Module

Files covered: `Crud.page.tsx`, `ItemTable.tsx`, `ItemModal.tsx`, `ItemForm.tsx`, `DeleteConfirmModal.tsx`, `StatusPlaceholder.tsx`, `Crud.module.tsx`

9.1 `Crud.page.tsx` — The Orchestrator

```
const CrudPage = () => {
  const {
    rows, yearOptions, isLoading, isFetching, isError, hasSearched,
    modalOpen, editCtx, deleteConfirmOpen,
    handleServerSearch, handleCreate, handleEdit,
    handleCloseModal, handleOpenDeleteConfirm, handleCloseDeleteConfirm,
  } = useCrudPage();

  const [pendingDeleteIds, setPendingDeleteIds] = useState<number[]>([]);

  const handleDeleteSelected = useCallback(
    (ids: number[]) => {
      setPendingDeleteIds(ids);
      handleOpenDeleteConfirm();
    },
    [handleOpenDeleteConfirm],
  );

  const handleDeleteCompleted = useCallback(() => setPendingDeleteIds([], []);

  return (
    <SingleColumnPage windowTitle="CRUD: Items">
      <PageTitle>CRUD: Items</PageTitle>
      <PageContent>
        {isError && <StatusPlaceholder variant="error">Unable to load items.
</StatusPlaceholder>}
        {!isError && (
          <ItemTable
            rows={rows} isLoading={isLoading} isFetching={isFetching}
            hasSearched={hasSearched} yearOptions={yearOptions}
            onServerSearch={handleServerSearch} onCreate={handleCreate}
            onEdit={handleEdit} onDeleteSelected={handleDeleteSelected}
          />
        )}
        {editCtx && <ItemModal open={modalOpen} onClose={handleCloseModal}
editCtx={editCtx} />}
        <DeleteConfirmModal
          open={deleteConfirmOpen} onClose={handleCloseDeleteConfirm}
          selectedIds={pendingDeleteIds} onDelete={handleDeleteCompleted}
        />
      </PageContent>
    </SingleColumnPage>
  );
}
```

```
);
};
```

Why is the page so thin? One job: layout and conditional rendering. All logic is in `useCrudPage`. The hook is testable without rendering UI. The page is readable at a glance.

pendingDeleteIds — why in the page, not the hook? It bridges the table (which knows selected rows) and the delete modal (which needs the IDs). It's a page-level wiring concern — the hook doesn't need to know about it.

{editCtx && <ItemModal ... />} — `ItemModal` requires `editCtx` as a non-optional prop. Conditional rendering avoids handling `null` inside the modal. The tradeoff is losing the closing animation — acceptable here.

<DeleteConfirmModal> is always rendered — always in the DOM, controlled by `open` prop. This preserves the closing animation.

9.2 components/ItemTable.tsx — TanStack Table

Skeleton Component

```
function TableSkeletonRows() {
  return (
    <Table className="table-fixed" aria-label="Loading items">
      <TableHeader>
        <TableRow>
          {Array.from({ length: SKELETON_COLS }).map((_, i) => (
            <TableHead key={i}><Skeleton className="h-3 w-2/3 rounded-sm" />
          </TableHead>
          )))}
        </TableRow>
      </TableHeader>
      <TableBody>
        {Array.from({ length: SKELETON_ROWS }).map((_, rowIdx) => (
          <TableRow key={rowIdx} className="h-10">
            {Array.from({ length: SKELETON_COLS }).map((_, colIdx) => (
              <TableCell key={colIdx}>
                <Skeleton className={`h-4 rounded-sm ${SKELETON_WIDTHS[colIdx %
SKELETON_WIDTHS.length]}` />
              </TableCell>
            )))}
          </TableRow>
        )))}
      </TableBody>
    </Table>
  );
}
```

`Array.from({ length: N }).map((_, i) => ...)` — creates N elements and maps over them. `_` is the element value (ignored). Used to render N skeleton rows/columns without hardcoding them.

`SKELETON_WIDTHS[colIdx % SKELETON_WIDTHS.length]` — cycles through width classes using modulo. Creates varied widths that look more natural than uniform skeletons.

`aria-label="Loading items"` — WCAG: screen readers announce this when the skeleton table is focused.

Why a skeleton instead of a spinner? Skeletons show the approximate layout of loading content, reducing perceived loading time. A spinner gives no layout information.

TanStack Table Setup

```
const table = useReactTable({
  data: rows,
  columns,
  getCoreRowModel: getCoreRowModel(),
  getSortedRowModel: getSortedRowModel(),
  getFilteredRowModel: getFilteredRowModel(),
  getPaginationRowModel: getPaginationRowModel(),
  globalFilterFn: "includesString",
  filterFns: {
    filterBuilder: (row, columnId, filterValue) =>
      filterBuilderFn(row, columnId, filterValue, FILTERABLE_COLUMNS),
  },
  defaultColumn: { filterFn: "filterBuilder" as never },
  state: { sorting, globalFilter: searchValue, columnFilters, rowSelection },
  onSortingChange: setSorting,
  onGlobalFilterChange: setSearchValue,
  onColumnFiltersChange: setColumnFilters,
  onRowSelectionChange: setRowSelection,
  enableRowSelection: true,
  getRowId: (row) => String(row.id),
  initialState: {
    pagination: { pageSize: PAGE_SIZE },
    columnVisibility: DEFAULT_COLUMN_VISIBILITY,
  },
});
```

- `getCoreRowModel()` — required. Without this, TanStack Table has no row model and renders nothing.
- `getSortedRowModel()` — enables client-side sorting. Without this, clicking column headers does nothing.
- `getFilteredRowModel()` — enables client-side filtering. Without this, search has no effect.
- `getPaginationRowModel()` — enables pagination. Without this, all rows show on one page.
- `globalFilterFn: "includesString"` — case-insensitive substring matching for global search.
- `filterFns: { filterBuilder: ... }` — registers the custom filter function. Without this, `TableFilterBuilder` filters have no effect.
- `defaultColumn: { filterFn: "filterBuilder" as never }` — applies custom filter to all columns. `as never` bypasses TanStack Table's type restriction on custom filter names.

- **state: { ... }** — controlled state. TanStack Table reads these to compute displayed rows.
- **getRowId: (row) => String(row.id)** — stable row IDs. Without this, TanStack Table uses row index — which changes when rows are filtered/sorted, breaking row selection.
- **initialState** — sets starting state without making it controlled. **pageSize: PAGE_SIZE** and **columnVisibility: DEFAULT_COLUMN_VISIBILITY**.

Column Width Normalization

```
const totalSize = table.getVisibleFlatColumns().reduce((sum, col) => sum + col.getSize(), 0);
const getColWidth = (size: number) => `${((size / totalSize) * 100).toFixed(2)}%`;
```

TanStack Table **size** values are proportions, not percentages. **getColWidth** normalizes them based on currently visible columns. If the user hides a column, the remaining columns expand proportionally.

table-fixed CSS — without it, the browser calculates column widths based on content, causing layout shifts as data loads. **table-fixed** fixes widths via the **style={{ width: getColWidth(...) }}** attribute.

Selected IDs Memo

```
const selectedIds = useMemo(
  () => table.getSelectedRowModel().rows.map((r) => r.original.id),
  [table, rowSelection], // eslint-disable-line react-hooks/exhaustive-deps
);
```

rowSelection in deps — even though it's not directly referenced inside the memo, it's needed because **table.getSelectedRowModel()** reads internal table state that updates when **rowSelection** changes. Without it, the memo returns stale selected IDs.

Render Logic

```
<TableContent>
  {showSkeleton ? (
    <TableSkeletonRows />
  ) : !hasSearched ? (
    <TableEmpty>Select filters above and click Search to load data.</TableEmpty>
  ) : table.getRowModel().rows.length === 0 ? (
    <TableEmpty>No items found.</TableEmpty>
  ) : (
    <div className="overflow-x-auto overflow-y-auto md:max-h-[calc(100vh-280px)]">
      <Table className="table-fixed" aria-label="Items list">
        <TableHeader>
          {table.getHeaderGroups().map((headerGroup) => (
            <TableRow key={headerGroup.id}>
              {headerGroup.headers.map((header) => (
                <TableHead key={header.id} style={{ width:
getColWidth(header.getSize()) }}>
```

```

        {header.isPlaceholder ? null :
flexRender(header.column.columnDef.header, header.getContext())}
      </TableHead>
    )))
  </TableRow>
  )))
</TableHeader>
<TableBody>
  {table.getRowModel().rows.map((row) => (
    <TableRow key={row.id} data-state={row.getIsSelected() && "selected"}>
      {row.getVisibleCells().map((cell) => (
        <TableCell key={cell.id} style={{ width:
getColWidth(cell.column.getSize()) }}>
          {flexRender(cell.column.columnDef.cell, cell.getContext())}
        </TableCell>
      ))}
    </TableRow>
  ))}
</TableBody>
</Table>
</div>
  )}
</TableContent>

```

- **flexRender(header.column.columnDef.header, header.getContext())** — TanStack Table's utility that renders a column's **header** definition. It handles both function headers (like `{ column } => <TableHeaderButton .../>`) and plain string headers.
- **header.isPlaceholder** — true for merged header cells in multi-level headers. We render **null** for these.
- **table.getRowModel().rows** — only the rows for the current page (after sorting, filtering, pagination).
- **data-state={row.getIsSelected() && "selected"}** — Shadcn's **TableRow** applies a highlight style when **data-state="selected"**.
- **md:max-h-[calc(100vh-280px)]** — on medium+ screens, the table scrolls vertically within a fixed height. Keeps the table visible without page-level scrolling.

9.3 components/ItemModal.tsx — Dialog + Form Wiring

Focus Management (WCAG)

```

const triggerRef = useRef<HTMLElement | null>(null);

useEffect(() => {
  if (open) {
    triggerRef.current = document.activeElement as HTMLElement;
  }
}, [open]);

```

useRef — stores a mutable value that persists across renders WITHOUT causing re-renders. Used here to remember the DOM element that had focus before the modal opened.

document.activeElement — the currently focused element. When the user clicks "Edit", the Edit button has focus. We capture it here.

```
<DialogContent onCloseAutoFocus={() => triggerRef.current?.focus()}>
```

onCloseAutoFocus — Shadcn's Dialog calls this when the dialog closes. We return focus to the element that triggered the modal. WCAG requirement: when a modal closes, focus must return to the trigger element so keyboard users don't lose their place.

The **useEffect** Reset

```
useEffect(() => {
  if (!open) return;
  reset(defaultValues);
}, [open, reset, defaultValues]);
```

useForm({ defaultValues }) only applies defaults on the first render. Without this **useEffect**, editing Student A, closing, then editing Student B would show Student A's data. The effect detects **open** changed to **true** and resets the form with the new defaults.

if (!open) return — don't reset when the modal closes. Without this guard, the form resets to empty values as the modal is closing — causing a flash of empty form before the animation completes.

Submit Handler

```
const onSubmit = useCallback(
  (values: ItemFormValues) => {
    const payload = buildPayload(values);

    if (isEdit && editCtx.studentActivityId) {
      updateMutation.mutate(
        { studentActivityId: editCtx.studentActivityId, studentId:
editCtx.studentId, studentKey: editCtx.studentKey, ...payload },
        { onSuccess: onClose }
      );
    } else {
      createMutation.mutate(
        { studentIds: [editCtx.studentId], selectedStudents: [{ studentId:
editCtx.studentId, studentKey: editCtx.studentKey }], ...payload },
        { onSuccess: onClose }
      );
    }
  },
  ,
```

```
[isEdit, editCtx, createMutation, updateMutation, onClose],
);
```

values: ItemFormValues — RHF passes validated, typed form values. These have already passed Zod validation — trust their types.

buildPayload(values) — extracts shared fields between create and update. `Number(values.activityId)` converts the Select's string back to a number for the API.

mutation.mutate(request, { onSuccess: onClose }) — per-call `onSuccess` closes the modal after the mutation succeeds. This runs IN ADDITION to the `onSuccess` in `useCreateItem` (which shows a toast and invalidates the cache).

<FormProvider {...methods}> — spreads all form methods into React Context. `ItemForm` reads them via `useFormContext()` — no prop drilling.

form.handleSubmit(onSubmit) — RHF's submit handler. Runs Zod validation first. If validation fails, populates `formState.errors` and does NOT call `onSubmit`. If validation passes, calls `onSubmit(validatedValues)`.

disabled={isSubmitting} — disables the submit button while the mutation is in flight. Prevents double-submits.

9.4 components/ItemForm.tsx — Form Fields

Self-Fetching Data (Component Data Ownership)

```
const { data: categoriesResponse, isLoading: categoriesLoading } =
  useGetCategories();
const categories = categoriesResponse?.ResponseActivities ?? [];
```

`ItemForm` fetches its own dropdown data. The page hook does NOT fetch categories and pass them down as props. React Query returns cached data instantly — no extra network request. This is the **component data ownership** principle.

useFormContext

```
const {
  control,
  watch,
  formState: { errors, submitCount },
} = useFormContext<ItemFormValues>();
```

Reads the form instance from `FormProvider` context. No props needed. `<ItemFormValues>` gives TypeScript autocomplete on field names.

watch("notes") — subscribes to the **notes** field. When **notes** changes, the component re-renders. Used to conditionally show the "Restrict Notes" checkbox:

```
const notes = watch("notes");
{!!notes && <Controller name="restrictedNotes" ... />}
```

submitCount — how many times the form has been submitted. Used to show the error summary only after the user has tried to submit:

```
const showErrorMessage = submitCount > 0 && Object.keys(errors).length > 0;
```

Controller for Shadcn Inputs

```
<Controller
  name="activityId"
  control={control}
  render={({ field }) => (
    <Select
      value={field.value ? String(field.value) : undefined}
      onChange={val => field.onChange(val)}
    >
      ...
    </Select>
  )}
/>
```

Shadcn's **Select**, **Checkbox**, **Calendar** are NOT standard HTML inputs — they don't work with RHF's **register()**. **Controller** provides a **value/onChange** interface that works with any custom component.

field.value ? String(field.value) : undefined — if the value is "" or 0 (sentinel), pass **undefined** so the Select shows the placeholder.

Accessible Error Messages

```
<SelectTrigger
  aria-invalid={!!errors.activityId}
  aria-describedby={errors.activityId ? "activityId-error" : undefined}
>
{errors.activityId && (
  <p id="activityId-error" className="text-sm text-destructive mt-1">
    {errors.activityId.message}
  </p>
)}
</SelectTrigger>
```

- **aria-invalid** — tells screen readers the field has an error
- **aria-describedby** — links the field to its error message by ID
- **id="activityId-error"** — the ID that **aria-describedby** references

This is the WCAG standard for accessible form validation. Screen readers read the error message when the field is focused.

9.5 components/DeleteConfirmModal.tsx — Safe Multi-Delete

```
const handleConfirm = async () => {
  if (selectedIds.length === 1) {
    await deleteMutation.mutateAsync({ studentActivityId: selectedIds[0] });
  } else {
    const results = await Promise.allSettled(
      selectedIds.map((id) => deleteMutation.mutateAsync({ studentActivityId: id
    )))
    );
    const failed = results.filter((r) => r.status === "rejected").length;
    if (failed > 0) toast.error(`${failed} of ${selectedIds.length} deletions
failed.`);
  }
  onClose();
  onDelete?.();
};
```

Promise.allSettled vs Promise.all:

- **Promise.all** — if ANY deletion fails, it rejects immediately and cancels the rest. Delete 5 items, item 3 fails → items 4 and 5 are never deleted.
- **Promise.allSettled** — waits for ALL deletions regardless of success or failure. Counts failures and reports them. Partial success is better than no success.

mutateAsync — returns a Promise that can be passed to **Promise.allSettled**. **mutate** doesn't return a Promise.

9.6 components/StatusPlaceholder.tsx — Accessible Empty States

```
<div role="status" aria-live="polite" className="flex flex-col items-center
justify-center py-16 gap-4">
  {icons[variant]}
  <p className="text-sm text-muted-foreground text-center">{children}</p>
</div>
```

- **role="status"** — WCAG landmark. Tells screen readers this region contains status information.
- **aria-live="polite"** — screen readers announce content changes when the user is idle.
"assertive" would interrupt immediately — only use for critical errors.

9.7 `Crud.module.tsx` — Lazy Loading and Routing

```
// Crud.module.tsx
export default function CrudModule() {
  return (
    <Routes>
      <Route index element={<CrudPage />} />
    </Routes>
  );
}

// main.tsx
const CrudModule = React.lazy(() => import("./features/react-crud-
v4/Crud.module"));

{
  path: "crud/*",
  element: (
    <Suspense fallback={<LoadingIndicator type="bar" size="sm" />}>
      <CrudModule />
    </Suspense>
  ),
}
```

`React.lazy`

`React.lazy(() => import("./path"))` defers loading the module's JavaScript bundle until the user navigates to that route. Without lazy loading, ALL feature code is bundled into one large file that the user downloads before seeing anything. With lazy loading, each feature is a separate chunk downloaded on demand — faster initial page load.

`<Suspense fallback={...}>`

While the lazy module is downloading, React renders the `fallback`. Without `<Suspense>`, React throws an error when it encounters a lazy component that hasn't loaded yet.

`path: "crud/*"` and `<Route index>`

The `*` in the parent route means "match anything after `/crud/`". The module's `<Route index>` matches `/crud/` exactly. To add a detail page, add `<Route path="detail/:id" element={<DetailPage />} />` inside the module — no changes to `main.tsx` needed.

Why a separate module file?

The module file is the **lazy-load boundary**. Everything imported by `Crud.module.tsx` is included in the lazy chunk. If `main.tsx` imported `CrudPage` directly (without lazy), it would be in the main bundle. The module file is intentionally thin — only routing. All real code is in `Crud.page.tsx` and its children.

Next: [Chapter 10 — Common Mistakes and Glossary](#)